

Documentation développeur

Table des matières

1. [Description du travail fourni](#)
 - 1.1 [Fonctionnalités implémentées](#)
 - 1.2 [Réponses aux 4 défis géographiques](#)
 - 1.3 [Fonctionnalités non implémentées](#)
 2. [Architecture du projet](#)
 3. [Exécution et lancement](#)
 4. [Architecture backend](#)
 - 4.1 [Routes](#)
 - 4.2 [Controller](#)
 - 4.3 [Service](#)
 - 4.4 [Repository](#)
 5. [Architecture frontend](#)
 - 5.1 [App.vue](#)
 - 5.2 [Composable](#)
 - 5.3 [Service](#)
 - 5.4 [RestaurantForm.vue](#)
 - 5.5 [RestaurantList.vue](#)
 6. [Résumé](#)
-

DESCRIPTION DU TRAVAIL FOURNI

1. Fonctionnalités implémentées

Frontend (Vue.js 3)

- Formulaire d'ajout avec les champs : nom, adresse, latitude, longitude, type de cuisine (liste déroulante), numéro de téléphone (optionnel)
- Validation côté client sur chaque champ avec messages d'erreur ciblés
- Liste des restaurants en tableau avec toutes les informations
- Coordonnées affichées avec 6 décimales (précision au décimètre)
- Badge coloré par type de cuisine avec emoji
- Recherche par nom ou adresse (insensible à la casse, avec délai de 300ms)
- Filtre par type de cuisine
- Pagination : 10 résultats par page (configurable, max 100)

Backend (Node.js + Express)

- `POST /api/restaurants` — créer un restaurant
- `GET /api/restaurants` — liste paginée
- `GET /api/restaurants/search?q=term` — recherche par nom ou adresse
- `GET /api/restaurants/filter?cuisine=type` — filtre par type de cuisine
- Validation des coordonnées côté serveur avec messages d'erreur explicites
- Gestion centralisée des erreurs (AppError + codes PostgreSQL)

Base de données (PostgreSQL)

- Table `restaurants` avec le schéma fourni dans le sujet
- Colonnes `DECIMAL(10,8)` et `DECIMAL(11,8)` pour les coordonnées (précision au millimètre)
- Contraintes `CHECK` sur latitude et longitude comme filet de sécurité

2. Réponses aux 4 défis géographiques du sujet

Défi	Réponse technique
Précision et validation des coordonnées	Trois niveaux en cascade : validation client (<code>RestaurantForm.vue</code>), validation serveur (<code>restaurantService.ts</code>), contrainte <code>CHECK</code> PostgreSQL comme filet de sécurité
Stockage en base de données	Colonnes <code>DECIMAL(10,8)</code> et non <code>FLOAT</code> pour éviter les dérives d'arrondi, précision au millimètre. Conversion <code>DECIMAL</code> → <code>number</code> via <code>parseCoords()</code> dans le repository
Affichage convivial	Coordonnées affichées avec <code>toFixed(6)</code> : 6 décimales fixes, précision au décimètre, cohérent visuellement
Validation des entrées	Chaque champ de coordonnée est validé indépendamment avec un message ciblé. Le formulaire bloque l'envoi tant qu'un champ est invalide. Les erreurs serveur s'affichent en bannière

3. Fonctionnalités non implémentées (bonus)

Par faute de temps, je n'ai pas implémenté les fonctionnalités bonus (carte Leaflet, calcul de distance, édition d'un restaurant).

ARCHITECTURE DU PROJET

```
geoptis/
├── package.json
├── .gitignore
├── README.md
├── MANUEL_TECHNIQUE.md
├── backend/
│   ├── app.ts
│   ├── db.ts
│   ├── schema.sql
│   ├── package.json
│   └── tsconfig.json
│   ├── public/
│   │   ├── index.html
│   │   ├── favicon.svg
│   │   └── assets/
│   ├── models/
│   │   └── restaurant.model.ts
│   ├── errors/
│   │   ├── AppError.ts
│   │   └── index.ts
│   ├── middleware/
│   │   ├── asyncHandler.ts
│   │   └── errorHandler.ts
│   ├── routes/
│   │   └── restaurants.ts
│   ├── controllers/
│   │   └── restaurantController.ts
│   ├── services/
│   │   └── restaurantService.ts
│   └── repositories/
│       └── restaurantRepository.ts
└── frontend/
    ├── index.html
    ├── vite.config.ts
    ├── package.json
    ├── tsconfig.json
    └── src/
        ├── main.ts
        ├── style.css
        ├── App.vue
        ├── models/
        │   └── restaurant.model.ts
        ├── services/
        │   └── restaurant.service.ts
        ├── composables/
        │   └── useRestaurants.ts
        └── components/
            ├── RestaurantForm.vue
            └── RestaurantList.vue
```

← scripts racine (dev, build, start)

← point d'entrée : configuration Express + démarrage serveur
← pool de connexions PostgreSQL
← schéma de la table restaurants

← frontend compilé par Vite, servi par Express

← JS et CSS minifiés (généré automatiquement)

← interfaces TypeScript partagées entre les couches

← classe d'erreur personnalisée `AppError<T>`
← helpers : `badRequest()`, `notFound()`, `serverError()`

← wrapper async → évite les try/catch dans les controllers
← gestionnaire d'erreurs centralisé (`AppError` + codes PostgreSQL)

← association URL → controller

← lecture req, appel service, envoi réponse

← validation + logique métier

← requêtes SQL uniquement

← point d'entrée Vue
← reset CSS global
← composant racine, assemble les composants

← interfaces TypeScript + constantes (couleurs, emojis)

← appels HTTP axios vers l'API

← état réactif centralisé + toutes les actions

← formulaire d'ajout + validation côté client
← tableau + recherche + filtre + pagination

EXÉCUTION ET LANCEMENT DE L'APPLICATION

1. Prérequis

- Node.js v16+
- PostgreSQL 16 installé et démarré (`brew services start postgresql@16` sur macOS)

2. Installation

```
git clone https://github.com/asad940/GE0PTIS.git
cd GE0PTIS
npm run setup
```

`npm run setup` installe les dépendances des trois dossiers en une seule commande (racine, backend, frontend).

3. Créer la base de données

```
npm run db:setup
```

Cette commande crée la base `geoptis` dans PostgreSQL et y exécute `backend/schema.sql`, ce qui crée la table `restaurants` avec toutes ses colonnes et contraintes.

4. Lancer le projet en développement

```
npm run dev
```

L'application est accessible sur <http://localhost:3000>

5. Scripts disponibles

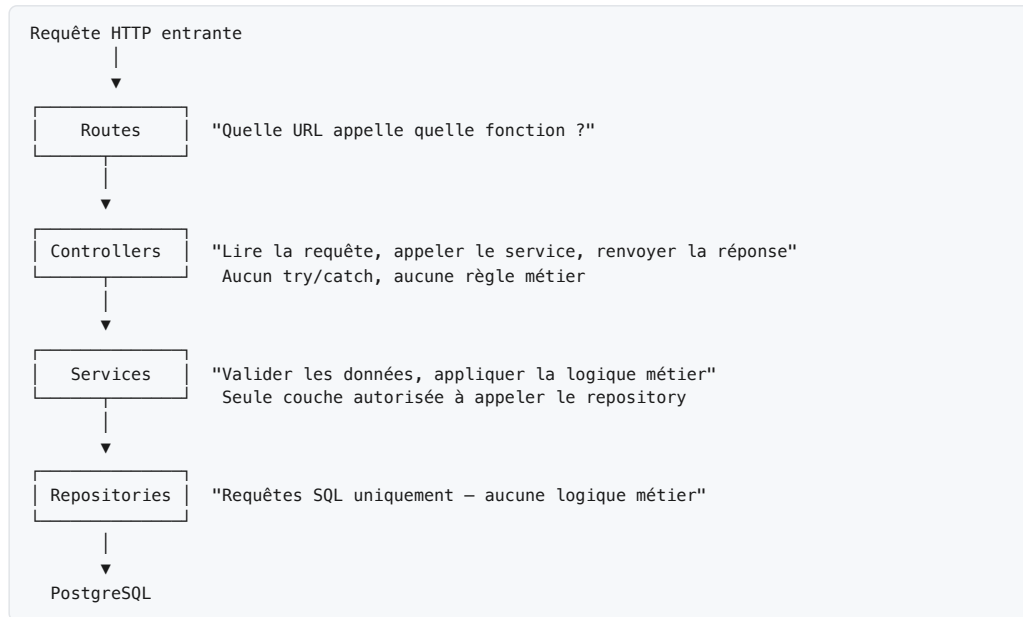
Commande	Description
<code>npm run setup</code>	Installe toutes les dépendances (racine, backend, frontend) en une commande
<code>npm run db:setup</code>	Crée la base de données et exécute le schéma SQL
<code>npm run dev</code>	Lance le backend et le frontend en parallèle
<code>npm run build</code>	Compile le frontend avec Vite
<code>npm run clean</code>	Supprime les <code>node_modules</code> , le build et les fichiers générés (utile avant de zipper le projet)

ARCHITECTURE BACKEND

Pour la partie backend, je suis parti sur une **architecture en couches** : un controller pour exposer les API, une couche service pour la logique métier (récupération des données, transformation en DTO afin de ne pas exposer directement la base de données), et une couche repository pour l'accès à la base. Chacune a une responsabilité unique et ne communique qu'avec la couche en dessous.

Cette approche respecte le **S de SOLID** (Single Responsibility Principle) : chaque fichier a une seule raison de changer. Modifier le SQL ne touche que le repository. Changer une règle de validation ne touche que le service.

J'ai choisi cette architecture parce que je l'avais déjà utilisée dans d'autres projets et je savais qu'elle fonctionnerait. Elle force à séparer les responsabilités dès le départ, ce qui est parfaitement adapté à ce contexte.



1. Routes

`routes/restaurants.ts`

Le fichier de routes a un seul rôle : **associer une URL à une fonction de controller**. Il ne contient aucune logique.

```
router.get('/search', controller.search); // ← déclaré AVANT /
router.get('/filter', controller.filter); // ← déclaré AVANT /
router.get('/', controller.getAll);
router.post('/', controller.create);
```

Les routes `/search` et `/filter` sont déclarées avant `/` car Express parcourt les routes dans l'ordre. Si `/` était en premier, Express interpréterait `/search` comme `/:id` avec `id = "search"`.

2. Controller

`controllers/restaurantController.ts`

Le controller a un rôle précis : **lire les paramètres de la requête, appeler le service, renvoyer la réponse**. Rien d'autre. Il ne contient aucune logique métier, aucun `try/catch`, aucune requête SQL.

```
export const create = asyncHandler(async (req: Request, res: Response) => {
  const restaurant = await service.createRestaurant(req.body);
  res.status(201).json(restaurant); // HTTP 201 = "Created"
});
```

Chaque fonction est enveloppée dans `asyncHandler`, voir la section 3.3 Gestion des erreurs pour l'explication.

3. Service

`services/restaurantService.ts`

Le service est la **couche la plus importante** : c'est là que vit toute la logique métier et la validation. C'est la seule couche autorisée à appeler le repository.

```
export async function createRestaurant(data: Partial<CreateRestaurantDto>): Promise<Restaurant> {
  validateRestaurantData(data); // 1. valider les champs
  const { lat, lng } = validateCoordinates(data.latitude, data.longitude); // 2. valider coords
  return repo.create({ ...data, latitude: lat, longitude: lng }); // 3. insérer en base
}
```

3.1 Modèles — `models/restaurant.model.ts`

Pour ne pas exposer directement la structure de la base au client, on utilise deux interfaces distinctes :

- **Restaurant** : représente un restaurant tel qu'il existe en base (`id`, `created_at` inclus) — c'est ce que le repository retourne.
- **CreateRestaurantDto** : représente ce que le client envoie pour créer un restaurant — sans `id` ni `created_at`, car ils sont générés par PostgreSQL. **DTO** = *Data Transfer Object*.

```
// Ce que le client envoie
interface CreateRestaurantDto {
  name: string;
  address: string;
  latitude: number;
  longitude: number;
  cuisine_type: CuisineType;
  phone_number?: string; // optionnel
}

// Ce que la base retourne
interface Restaurant extends CreateRestaurantDto {
  id: number; // généré par PostgreSQL
  created_at: Date; // généré automatiquement
}
```

3.2 Types génériques

J'ai défini `PaginatedResult<T>` et `AppError<T>` comme des **types génériques** : le `<T>` signifie que le type de données n'est pas fixé à la compilation.

PaginatedResult<T> — aujourd'hui utilisé avec `Restaurant`, mais si demain on ajoute des utilisateurs ou des avis, la même interface fonctionne sans modification :

```
PaginatedResult<Restaurant> // aujourd'hui
PaginatedResult<User>       // demain, sans toucher l'interface
```

C'est le **O de SOLID** (Open/Closed Principle) : l'interface est **ouverte à l'extension** pour n'importe quel type futur, mais **fermée à la modification** : on n'a jamais besoin d'y toucher.

AppError<T> — permet d'attacher des données contextuelles optionnelles à l'erreur, comme une liste de champs invalides :

```
throw new AppError<string[]>(400, 'Données invalides', ['latitude', 'longitude'])
```

Sans générique, on serait forcé d'utiliser `any`, ce qui supprime toute sécurité TypeScript sur ces données.

3.3 Gestion des erreurs — `errors/` et `middleware/`

Pour éviter les `try/catch` répétés dans chaque controller, j'ai mis en place une **exception personnalisée** `AppError`, pour quatre raisons :

1. **Transporter le code HTTP avec l'erreur** — `Error` native ne contient que `message`. Impossible de savoir si c'est un 400, 404 ou 500. `AppError` transporte les deux.
2. **Centraliser la gestion** — sans `AppError`, chaque controller devrait écrire son propre `try/catch`. Avec `AppError` + `asyncHandler`, aucun controller ne gère d'erreur.
3. **Distinguer nos erreurs des erreurs inattendues** — `errorHandler` détecte avec `instanceof AppError` si l'erreur est volontaire (on renvoie notre message) ou inattendue (on renvoie un message générique).
4. **Rendre les erreurs user-friendly** — le client reçoit toujours un message compréhensible, jamais un message brut PostgreSQL.

Deux dossiers complémentaires assurent ce mécanisme :

`errors/` — définit ce que sont les erreurs :

- `AppError.ts` : classe personnalisée qui transporte un **code HTTP** avec le message. En étendant `Error`, Express la reconnaît nativement.
- `index.ts` : helpers lisibles — `badRequest()`, `notFound()`, `serverError()` — qui permettent d'écrire `throw badRequest('message')` dans le service.

```
// Dans le service — une seule ligne, aucun try/catch
if (latitude < -90 || latitude > 90)
  throw badRequest('La latitude doit être entre -90 et 90');
```

`middleware/` — traite les erreurs dans Express :

- `asyncHandler.ts` : wrapper qui attrape les erreurs des fonctions `async` et les transmet à Express via `next(err)`. Sans lui, une erreur dans un controller async serait silencieuse.
- `errorHandler.ts` : middleware centralisé enregistré en dernier dans `app.ts`. Il reçoit toutes les erreurs, distingue les `AppError` volontaires des erreurs PostgreSQL, et renvoie la bonne réponse HTTP.

```
SERVICE      → throw badRequest('...')
ASYNC HANDLER → .catch(next) → next(AppError)
ERROR HANDLER → res.status(400).json({ error: '...' })
CLIENT       → reçoit le message d'erreur
```

`errorHandler` intercepte également les erreurs internes que **PostgreSQL renvoie lui-même** quand une contrainte est violée. Sans interception, le client recevrait le message brut, incompréhensible :

```
// Message brut PostgreSQL
'new row for relation "restaurants" violates check constraint "valid_latitude"'

// Message après interception dans errorHandler
'Coordonnées géographiques hors limites (latitude: -90/90, longitude: -180/180)'
```

Code PG	Situation	Réponse renvoyée
23514	Contrainte <code>CHECK</code> violée (coordonnées hors limites)	HTTP 400
23502	Colonne <code>NOT NULL</code> reçoit <code>null</code> (champ obligatoire manquant)	HTTP 400
23505	Doublon sur colonne <code>UNIQUE</code>	HTTP 409
22001	Valeur trop longue pour un <code>VARCHAR(n)</code>	HTTP 400

3.4 Ne jamais faire confiance au client

La validation côté client (dans le formulaire Vue) peut être contournée, en désactivant JavaScript ou en envoyant une requête directement avec `curl`.

Sans validation serveur, ces données invalides arriveraient en base. C'est pourquoi la validation existe à trois niveaux :

Niveau	Fichier	Rôle
1. Client	<code>RestaurantForm.vue</code> — <code>validate()</code>	Confort utilisateur, réponse immédiate sans aller-retour réseau
2. Service	<code>restaurantService.ts</code> — <code>validateRestaurantData()</code> + <code>validateCoordinates()</code>	Sécurité applicative — bloque toute donnée invalide, quelle que soit la source
3. Base	Contraintes <code>CHECK</code> dans <code>schema.sql</code>	Filet ultime — même si le service est contourné

La validation côté client est pour le **confort**. La validation côté serveur est pour la **sécurité**.

4. Repository

`repositories/restaurantRepository.ts`

Le repository a un seul rôle : **écrire et lire en base de données**. Aucune validation, aucune logique métier, uniquement du SQL.

J'ai choisi de ne pas utiliser d'ORM (Prisma, Sequelize) : étant donné la simplicité de l'application, ce serait une dépendance inutile. Le SQL direct donne un contrôle total sur les requêtes et rend le code plus performant, sans couche d'abstraction entre le code et PostgreSQL.

4.1 Schéma de base de données

```
CREATE TABLE restaurants (  
  id          SERIAL PRIMARY KEY,  
  name        VARCHAR(255) NOT NULL,  
  address     VARCHAR(500) NOT NULL,  
  latitude    DECIMAL(10,8) NOT NULL,  
  longitude   DECIMAL(11,8) NOT NULL,  
  cuisine_type VARCHAR(50)  NOT NULL,  
  phone_number VARCHAR(20),  
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT valid_latitude CHECK (latitude >= -90 AND latitude <= 90),  
  CONSTRAINT valid_longitude CHECK (longitude >= -180 AND longitude <= 180)  
);
```

J'ai utilisé `DECIMAL` et non `FLOAT` pour stocker les coordonnées. `FLOAT` stocke les nombres avec une approximation binaire et peut introduire des dérives invisibles (`48.8529` stocké comme `48.852899999...`). `DECIMAL(10,8)` garantit exactement 8 décimales, soit une précision d'environ **1 mm** sur le terrain, indispensable pour des données géospatiales.

La validation des coordonnées est faite à deux niveaux :

- Dans le **service** (avant la requête SQL) → message d'erreur user-friendly
- Par les **contraintes PostgreSQL** (filet de sécurité côté base) → même si le service est contourné, la base refuse les données invalides

4.2 Particularité : `DECIMAL` retourné en string par le driver `pg`

Le driver `pg` retourne les colonnes `DECIMAL` sous forme de **string** au lieu de `number`. Sans correction, `latitude` vaudrait `"48.852900"` (string) alors que les modèles déclarent `number`.

J'ai choisi de faire la conversion **dans le repository**, au plus près de la source, via une fonction privée `parseCoords()` :

```
function parseCoords(row: any): Restaurant {
  return {
    ...row,
    latitude: parseFloat(row.latitude),
    longitude: parseFloat(row.longitude),
  };
}
```

Toutes les fonctions du repository (`findAll`, `search`, `filterByCuisine`, `create`) passent leurs résultats par `parseCoords()` avant de les retourner. Le reste de l'application reçoit donc toujours de vrais `number` .

```
export async function create(data: CreateRestaurantDto): Promise<Restaurant> {
  const result = await pool.query(
    `INSERT INTO restaurants (name, address, latitude, longitude, cuisine_type, phone_number)
    VALUES ($1, $2, $3, $4, $5, $6)
    RETURNING *`,
    [data.name, data.address, data.latitude, data.longitude, data.cuisine_type, data.phone_number]
  );
  return parseCoords(result.rows[0]);
}
```

`RETURNING *` retourne la ligne insérée directement, en une seule requête au lieu de faire un `INSERT` puis un `SELECT` .

J'ai bien sûr laissé PostgreSQL gérer l' `id` et non le client. Si le client choisissait son propre `id` , deux utilisateurs simultanés pourraient choisir le même et provoquer une collision, et un client malveillant pourrait cibler l' `id` d'un enregistrement existant pour l'écraser. `SERIAL PRIMARY KEY` règle les deux problèmes : PostgreSQL génère un entier unique et auto-incrémenté à chaque insertion.

4.3 Pagination

Sans pagination, charger 10 000 restaurants signifierait 10 000 lignes SQL + des mégaoctets de JSON pour afficher 10 lignes à l'écran.

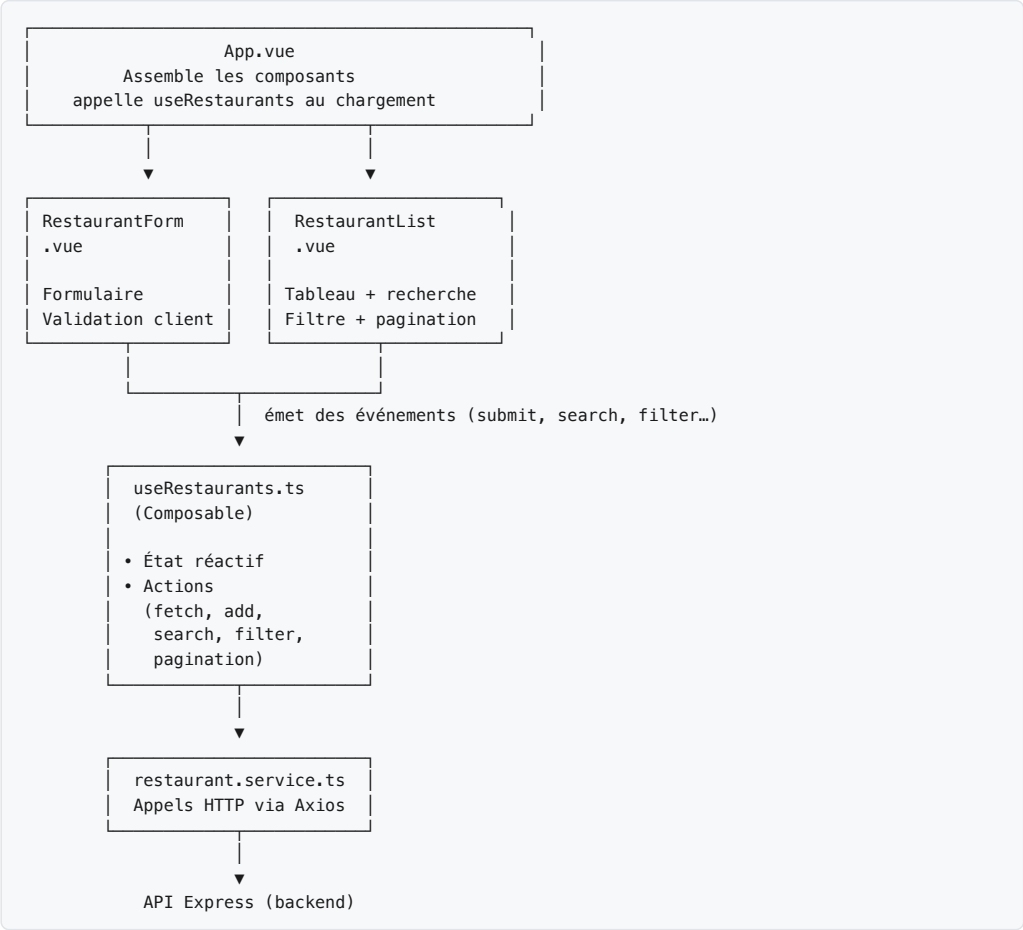
```
SELECT * FROM restaurants ORDER BY created_at DESC LIMIT 10 OFFSET 20
-- OFFSET = (page - 1) × limit → page 3 = sauter les 20 premiers
```

Le total et les données sont calculés **en parallèle** avec `Promise.all` pour ne pas faire deux requêtes séquentielles :

```
const [rowsResult, countResult] = await Promise.all([
  pool.query('SELECT * FROM restaurants LIMIT $1 OFFSET $2', [limit, offset]),
  pool.query('SELECT COUNT(*) FROM restaurants'),
]);
// Les deux requêtes partent en même temps → deux fois plus rapide qu'en séquence
```

ARCHITECTURE FRONTEND

Pour la partie frontend, j'ai appliqué le même principe de séparation des responsabilités qu'au backend : chaque fichier a un rôle unique et ne s'occupe que de ce qui le concerne.



Fichier	Rôle
App.vue	Assemble les composants, déclenche le chargement initial
components/RestaurantForm.vue	Formulaire d'ajout avec validation côté client
components/RestaurantList.vue	Tableau, barre de recherche, filtre par cuisine, pagination
composables/useRestaurants.ts	État réactif centralisé + toutes les actions
services/restaurant.service.ts	Appels HTTP Axios vers l'API — une fonction par endpoint
models/restaurant.model.ts	Interfaces TypeScript + constantes (couleurs et emojis par cuisine)

1. App.vue

App.vue est le composant racine. Il assemble RestaurantForm et RestaurantList, et coordonne les données entre eux via le composable useRestaurants. Il ne contient aucune logique métier, il délègue tout.

```
const { restaurants, paginated, loading, error, isFiltering,
        fetchRestaurants, addRestaurant, goToPage, applySearch, applyFilter, reset,
    } = useRestaurants();

onMounted(fetchRestaurants); // charge la liste au démarrage
```

`onMounted` est un hook Vue qui exécute une fonction après que le composant est affiché pour la première fois.

2. Composable — `composables/useRestaurants.ts`

Le composable regroupe tout l'état réactif et toutes les actions de l'application. Sans lui, `App.vue` contiendrait des dizaines de `ref()` et de fonctions mélangées avec le template, ce qui serait illisible. Les composants restent ainsi focalisés sur l'affichage, le composable sur la logique.

```
const paginated      = ref<PaginatedResult<Restaurant> | null>(null);
const filteredList   = ref<Restaurant[]>([]);
const loading         = ref(false);
const error           = ref<string | null>(null);
const isFiltering     = computed(() => search.value.trim() !== '' || cuisineFilter.value !== '');
```

`isFiltering` est un `computed` : il recalcule automatiquement sa valeur quand `search` ou `cuisineFilter` change. Il est utilisé pour masquer la pagination et afficher la bonne liste selon le contexte.

La fonction centrale `fetchRestaurants` décide quel endpoint appeler selon l'état :

```
async function fetchRestaurants() {
    if (search.value.trim()) filteredList.value = await service.search(search.value);
    else if (cuisineFilter.value) filteredList.value = await service.filter(cuisineFilter.value);
    else paginated.value = await service.getAll(page.value, limit.value);
}
```

3. Service

`services/restaurant.service.ts`

Seul fichier du frontend qui connaît les URLs de l'API. Toutes les autres couches passent par lui pour communiquer avec le backend.

J'ai choisi **Axios** plutôt que le `fetch` natif — c'est la librairie la plus utilisée dans les projets Vue et React, elle est parfaitement adaptée à une application simple comme celle-ci et évite beaucoup de code répétitif :

	fetch natif	Axios
Erreurs HTTP (400, 500...)	Ne rejette pas la promesse — à gérer manuellement	Rejette automatiquement si status ≥ 400
Lecture du JSON	<code>.json()</code> obligatoire	Automatique
Paramètres URL	À construire à la main	<code>{ params: { page, limit } } → ?page=1&limit=10</code>

```
const BASE = '/api/restaurants';

export const restaurantService = {
  getAll(page: number, limit: number) {
    return axios.get(BASE, { params: { page, limit } }).then(r => r.data);
  },
  search(q: string) {
    return axios.get(`${BASE}/search`, { params: { q } }).then(r => r.data);
  },
  filter(cuisine: string) {
    return axios.get(`${BASE}/filter`, { params: { cuisine } }).then(r => r.data);
  },
  create(data: CreateRestaurantDto) {
    return axios.post(BASE, data).then(r => r.data);
  },
};
```

4. RestaurantForm.vue

Formulaire d'ajout avec validation côté client. Il n'effectue pas d'appel API directement : il émet un événement `submit` au parent (`App.vue`) qui appelle `addRestaurant` . Le formulaire reste ainsi découplé de la logique réseau.

```
async function handleSubmit() {
  if (!validate()) return; // bloque l'envoi si un champ est invalide
  emit('submit', {
    name: form.name.trim(),
    latitude: parseFloat(form.latitude), // converti en number avant l'envoi
    // ...
  });
}
```

5. RestaurantList.vue

Tableau des restaurants avec recherche, filtre et pagination. La recherche utilise un **debounce** de 300ms pour éviter d'envoyer une requête API à chaque lettre tapée :

```
function onSearch() {
  clearTimeout(searchTimeout);
  searchTimeout = setTimeout(() => {
    emit('search', searchInput.value);
  }, 300); // attend 300ms d'inactivité avant d'émettre
}
```

Sans ce délai, taper "comptoir" (8 lettres) déclencherait 8 requêtes. Avec 300ms, une seule requête est envoyée quand l'utilisateur a fini de taper.

La recherche et le filtre sont **mutuellement exclusifs** : activer l'un réinitialise l'autre. La pagination est masquée en mode recherche/filtre car tous les résultats correspondants sont affichés directement.

RÉSUMÉ

Le backend est entièrement mon domaine d'études, j'ai pu y être à l'aise et développer une architecture robuste et modulable en respectant au maximum les principes SOLID, la sécurité et la séparation des responsabilités. C'est la partie que j'ai le plus appréciée : elle m'a permis de consolider mes compétences de développeur et de mettre en pratique des patterns que j'utilise au quotidien.

Pour la partie frontend, je me suis aidé de l'IA, la difficulté étant clairement du côté frontend pour moi. J'ai fait le maximum pour documenter les fonctions et les décisions techniques afin que le code soit lisible et compréhensible par n'importe quel développeur qui reprendrait le projet.